

Lab3

知识清单

1. 进程控制块

- 操作系统通过进程控制块（PCB）感知和管理进程，PCB与进程一一对应。
- PCB数据结构是Env结构体。
- 管理机制：
 - 进程控制块数组 `envs` 在系统启动时静态分配，数量有上限。
 - 空闲链表 `env_free_list`：使用 `LIST` 结构管理空闲（`ENV_NOT_RUNNABLE`）进程，分配时按倒序插入，使编号小的优先分配。
 - 调度队列 `env_sched_list`：使用 `TAILQ` 结构管理所有就绪（`ENV_RUNNABLE`）进程，支持在头部和尾部高效插入/取出。

2. 段地址映射

- 段地址映射函数 `map_segment`：在两级页表中建立虚拟地址段到物理地址段的映射，映射区间为 `[va, va + size)` 到 `[pa, pa + size)`。要求 `size` 为页面大小整数倍，并为页表项设置权限。

3. 进程的标识

- `env_id`：进程的标识符，在进程创建时被赋予。由 `mkenvid` 函数生成一个唯一的 `env_id`。
- `env_asid`：进程虚拟地址空间的标识。
 - ASID分配：`asid_alloc` 函数。
 - ASID作用：作为 TLB 标识的一部分，区分不同进程的虚拟地址空间，避免页表切换时清空整个 TLB，提升性能。
 - ASID管理：位图法管理，占据8个bit，最多可管理256个可用ASID。当ASID耗尽时创建进程，内核会发生崩溃。

4. 设置进程控制块

- 用户地址空间布局：
 - `ULIM` 以上为内核共享区（`kseg0/kseg1`），不经过 TLB 映射。
 - `UVPT` 到 `ULIM` 之间是进程自己的页表。
 - `UTOP` 到 `UVPT` 之间是所有进程共享的只读空间。
 - `UPAGES` 和 `UENVS` 处将内核的 `pages` 和 `envs` 数组映射到用户空间，供用户程序只读访问。

- `env_setup_vm`: 为新进程初始化页表, 复制模板页表 `base_pgdir` 中的内核共享映射, 并设置页目录自映射项。
- **上下文初始化**:
 - **状态寄存器**: 设置 `cp0_status`, 确保进程启动时处于**用户模式**、**中断开启**且指向正确的异常级别。
 - **栈指针 (sp)**: 初始化为 `USTACKTOP - sizeof(int) - sizeof(char **)`, 预留出存放 `argc` 和 `argv` 的空间, 同时为用户提供运行空间。
- `env_alloc`: 申请并初始化一个进程控制块。

5. 二进制镜像加载

`load_icode`负责加载可执行文件binary到进程的内存中。

- **解析 ELF 文件**:
 - 通过 `elf_from` 获取 ELF 文件头信息, 验证文件类型为可执行文件。
 - 遍历每个程序段 (program segment), 获取其虚拟地址 `va`、内存大小 `sgsize`、文件大小 `bin_size` 以及页面权限 `perm`。
- **调用段加载器**:
 - 对每个程序段, 调用 `elf_load_seg` 完成实际加载。
 - `elf_load_seg` 负责处理段内可能出现的非页对齐起始地址和尾部未满页, 按页粒度向回调函数 `map_page` 传递需要映射的页面信息。
- **回调函数实现页面操作**:
 - `load_icode_mapper` 作为 `map_page` 的具体实现, 为每个页面分配物理页框, 并在进程页表中建立虚拟地址到物理页的映射。
 - 若 `src` 非空, 则将对应的 ELF 数据拷贝到物理页中; 否则 (如 `.bss` 段未填充部分), 将页面剩余部分清零。

6. 进程创建

`env_create`分配一个新的Env结构体, 设置进程控制块, 并将程序载入到目标进程的地址空间。

- 调用 `env_alloc` 申请并初始化 PCB。
- 设置进程的优先级和状态。
- 调用 `load_icode` 将用户程序的二进制代码加载至该进程的地址空间。
- 将进程插入 **调度队列**。

7. 进程运行与切换

顶层调度函数`env_run`负责进程的运行和切换。

- **上下文保存**: 每个进程的上下文存储在 `Trapframe` 结构中。如果当前有正在运行的进程, 则保存将当前进程 `Trapframe`。

- **环境切换：**
 - 切换 `curenv` 为即将运行的进程。
 - 设置全局变量 `cur_pgdir` 为当前进程页目录的物理地址，用于 TLB 重填。
- **恢复现场：**调用 `env_pop_tf` 函数，恢复现场、异常返回。
- `RESTORE_ALL` 恢复寄存器，把 `Trapframe` 中保存的通用寄存器值恢复到 CPU 硬件寄存器中。
- 最后执行 `eret` 指令，原子性清除 `EXL` 位，跳转到 `EPC` 指向的用户地址。

8、MIPS 协处理器 CP0 与异常基础

- **CP0 核心寄存器：**
 - `BadVAddr` (8号)：当发生地址错异常或 TLB Miss 时，硬件自动记录引发故障的虚拟地址。
 - `Status` (12号)：状态寄存器，包含中断屏蔽位 (15-8位) 以及标识 CPU 运行状态的 `EXL`、`UM` 等位域。
 - `Cause` (13号)：记录导致异常的原因，15-8位记录已发生的硬件/软件中断，6-2位 (`ExcCode`) 记录具体的异常类型码。
 - `EPC` (14号)：记录异常处理结束后程序恢复执行的 PC 目标位置。
- **硬件处理异常的四个步骤：**
 - i. 将异常返回地址保存到 `EPC` 寄存器。
 - ii. 设置 `Status` 寄存器的 `EXL` 位，强制 CPU 进入内核态并全局禁止中断。
 - iii. 将异常原因及状态记录到 `Cause` 寄存器。
 - iv. 自动跳转到对应的异常入口物理地址处开始取指执行，后续交由软件处理。

9、异常分发机制 (Exception Dispatch)

- **异常入口地址：**在 MIPS 4Kc 中，链接器按要求将普通异常处理入口 (`.text.exc_gen_entry`) 放置在 `0x80000180`，用户态 TLB Miss 异常入口 (`.text.tlb_miss_entry`) 放置在 `0x80000000`。
- **现场保存 (SAVE_ALL)：**进入异常处理程序后，首先通过 `SAVE_ALL` 宏判断异常是否发生于内核态 (异常重入)。随后正确设置 `sp` 指向内核异常栈，并将 CPU 上下文保存到 `Trapframe` 结构中。
- **分发流程 (`exc_gen_entry`)：**
 - i. 清除 `Status` 寄存器中的 `UM`、`EXL`、`IE` 位，保持内核态且允许嵌套异常。
 - ii. 提取 `Cause` 寄存器中的 `ExcCode` (第 2~6 位) 作为异常码。
 - iii. 使用该异常码作为索引，查找 `exception_handlers` 数组，并跳转至对应的具体处理函数。

10、异常向量组 (Exception Handlers)

- **异常处理映射表：**内核通过 `exception_handlers` 数组记录不同异常码对应的处理函数。
- **常用 MIPS 异常定义：**
 - **0号异常：**中断 (如时钟中断、控制台中断)，对应 `handle_int`。

- **1号异常**：存储异常（只读页写入），对应 `handle_mod`。
- **2号异常**：TLB Load 异常，对应 `handle_tlb`。
- **3号异常**：TLB Store 异常，对应 `handle_tlb`。
- **8号异常**：系统调用（执行 `syscall` 触发），对应 `handle_sys`。

11、时钟中断机制

- **CP0 Timer 原理**：MIPS CP0 内置 Timer，由不断按固定频率自增的 `Count` 寄存器和保持不变的 `Compare` 寄存器组成。当二者相等且非0时，触发 7 号硬件中断。
- **中断开启与重置**：通过 `RESET_KCLOCK` 宏将 `Count` 清零并重新配置 `Compare` 以设定下一次中断周期。在每次进程调度（`env_pop_tf`）时重设时钟并开启中断响应。
- **处理链路**：触发中断 -> 跳转 `0x80000180` -> 分发至 `handle_int` -> 识别出 7 号中断源 -> 执行 `timer_irq` -> 进入 `schedule` 函数进行进程调度。

12、进程轮转调度（Round-Robin Scheduling）

- **时间片量化**：MOS 使用时钟中断间隔（`TIMER_INTERVAL`）来衡量时间片，进程的优先级（`env_pri`）即为其时间片的乘数。
- **调度队列**：就绪态（`ENV_RUNNABLE`）的进程均通过 `env_sched_link` 维护在调度链表（`env_sched_list`）中。
- **触发切换的条件**：尚未调度过进程、当前进程时间片耗尽、当前进程阻塞或退出、或主动调用 `yield` 强制切换。
- **调度执行**：若需切换，将当前就绪进程移至队列尾部，取出队首进程重置时间片；最终调用 `env_run` 不返回地跳转执行目标进程。

13、TLB Miss 异常的软件重填（Software Refill）

- **触发场景**：硬件 MMU 发出虚拟地址进行访存，当 TLB 中无匹配映射项时，硬件中止访存并触发 TLB Miss 异常。
- **硬件记录现场与软硬件协同拼接**：MIPS CPU 自动将出错的虚拟地址填入 `BadVAddr` 寄存器。特别注意 **ASID 由软件在进程调度切换时提前填入**，二者自动拼成完整的 `<VPN, ASID>` 键。同时，`Cause` 记录 TLBL 或 TLBS 异常码，并跳转至 `0x80000000`。
- **软件处理流程**：
 - i. 进入 `tlb_miss_entry`，最终调用 `do_tlb_refill`。
 - ii. 获取 `BadVAddr` 中的地址，在当前进程页目录（`cur_pgdir`）中逐级查表。
 - iii. 找到对应的物理页号（PFN）和权限位，填入 CP0 的 `EntryLo` 寄存器。
 - iv. 执行 `tlbwr` 指令，利用 `EntryHi`（已包含硬件填入的 VPN 和软件填入的 ASID）和 `EntryLo` 中的完整映射信息随机写入 TLB 中。
 - v. 调用 `ret_from_exception` 恢复现场并返回，CPU 重新执行刚才引发 Miss 的访存指令。

练习表单

请阅读指导书，完成下列练习：

练习序号	练习名	练习所在文件
3.1	env-init	kern/env.c
3.2	map-segment	kern/env.c
3.3	env-setup-vm	kern/env.c
3.4	env-alloc	kern/env.c
3.5	load-icode-mapper	kern/env.c
3.6	load-icode	kern/env.c
3.7	env-create	kern/env.c
3.8	env-run	kern/env.c
3.9	exc-entry	kern/entry.S
3.10	exc-entry-lds	kernel.lds
3.11	reset-kclock-macro	include/kclock.h
3.12	sched	kern/sched.c

测试点情况

本题共有 4 个测试点。

测试点分数分布

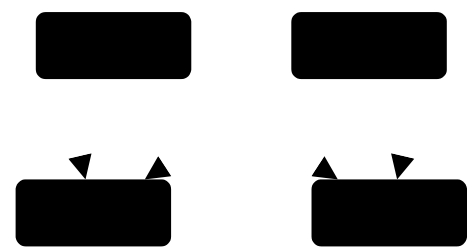
测试点编号	测试内容	本地测试（调试模式）	本地测试（开启优化）	分数
3_1	进程控制块管理测试	make test lab=3 1 && make run	MOS PROFILE=release make test lab=3_1 && make run	20
3_2	加载进程到内存测试	make test lab=3 2 && make run	MOS PROFILE=release make test lab=3_2 && make run	20

测试点编号	测试内容	本地测试（调试模式）	本地测试（开启优化）	分数
3_3	按优先级轮转运行进程测试	make test lab=3 3 && make run	MOS PROFILE=release make test lab=3_3 && make run	30
3_4	复杂程序运行测试	make test lab=3 4 && make run	MOS PROFILE=release make test lab=3_4 && make run	30

重要提示：

- 在提交评测前，请同学们充分进行本地测试；只要通过了以上两种模式的本地测试，在线评测也将通过。
- 在线评测时，所有的 .mk 文件、所有的 Makefile 文件、init/init.c、以及 tools/ 目录下无需填空的文件将被覆盖为标准版本，因此请同学们本地开发时，不要在这些文件中编写实际功能所依赖的代码，避免“本地测试通过，但在线评测不通过”的情况。
- 请善用 qemu 调试（单步执行），make objdump 等工具（反汇编）。

测试点依赖关系（箭头方向表示依赖方向）



测试点输出

重要提醒：具体输出逻辑请看 tests/ 下对应的测试程序，以下展示的输出仅供参考。

测试点 3_1